

MODULE 4 · NETWORKING

Network Policies

Pod-level firewalling: deny by default, allow by exception

Without a policy, every Pod can reach every Pod

The cluster network is **flat by default**: any Pod can talk to any other Pod, in any namespace, on any port. That's great for getting started and terrible for a shared cluster.

A **NetworkPolicy** selects Pods by label and, from that point on, only allows the traffic it explicitly lists — everything else to/from those Pods is dropped.

BY THE END YOU CAN

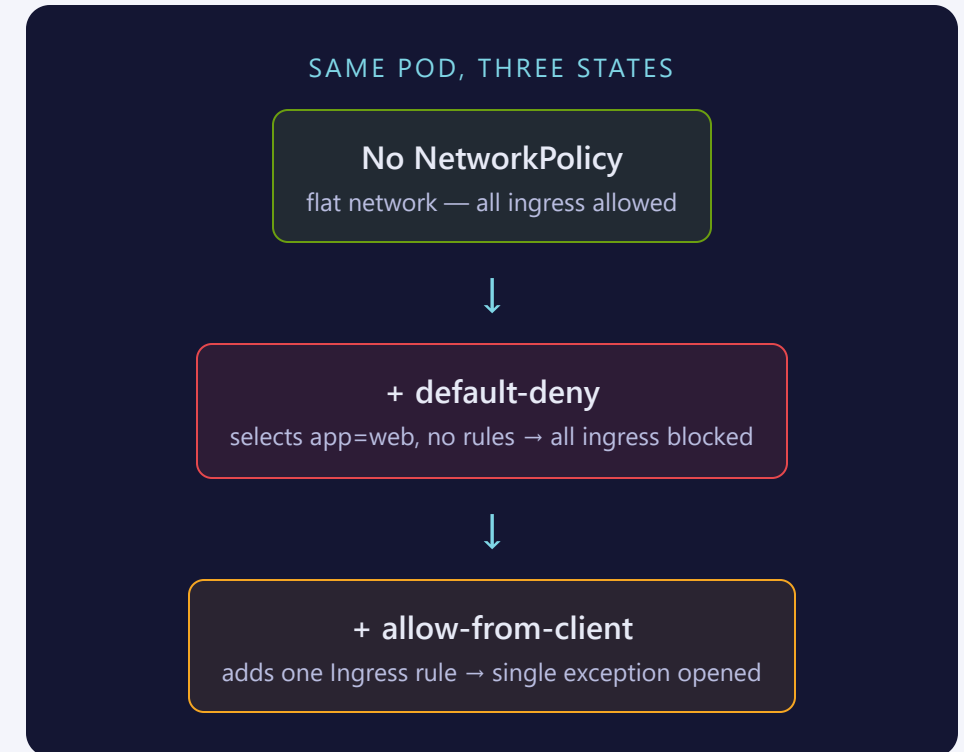
- Explain why "no policy" means "no isolation"
- Write a default-deny + an allow-from-app rule
- Tell ingress from egress
- Spot the default-deny-breaks-DNS gotcha

CONCEPT

Selecting a Pod flips its default from allow to deny

A NetworkPolicy's podSelector picks which Pods it governs. The moment **any** policy selects a Pod for a traffic direction (Ingress or Egress), that direction becomes **deny-by-default** for that Pod — only what the policy's rules list gets through.

Policies are **additive**: there is no explicit "deny" rule. You deny by selecting a Pod and simply not allowing something; a second policy over the same Pod adds more allowed traffic, it never removes any.



Two independent directions, two independent switches

DIRECTION	GOVERNS	RULE FIELD	ONCE SELECTED
Ingress	Traffic into the selected Pods	ingress: - from:	only listed sources may reach the Pod
Egress	Traffic out of the selected Pods	egress: - to:	the Pod may only reach listed destinations

PODSELECTOR VS NAMESPACESELECTOR

Both narrow "from"/"to" by label. Combined in one entry they **AND** together (same namespace AND that label); as separate list entries they **OR** (either source is allowed). `policyTypes` lists which directions a policy touches — listing only Ingress leaves Egress fully open, even from the same policy object.

Deny by selecting, then leave no rule to match

netpol-default-deny.yaml

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: web-default-deny
  namespace: training
spec:
  podSelector:
    matchLabels:
      app: web
  policyTypes:
    - Ingress
```

WATCH

podSelector picks the Pods it governs; an empty ingress: [] with policyTypes: [Ingress] is a **full deny** — that's the default-deny pattern, not a mistake.

A second, additive policy punches exactly one hole

netpol-allow-from-client.yaml

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: web-allow-from-client
  namespace: training
spec:
  podSelector:
    matchLabels:
      app: web
  policyTypes:
    - Ingress
  ingress:
    - from:
        - podSelector:
            matchLabels:
              app: client-allowed
  ports:
    - protocol: TCP
      port: 80
```

`ingress[ ].from` selects senders by `podSelector/namespaceSelector`; this is a **separate, additive** object — it adds an exception on top of the deny, it never replaces it.

There's no shortcut — but you can validate first

IMPERATIVE

No `kubectl create networkpolicy` — the rule graph only exists as YAML

DECLARATIVE

`kubectl apply -f netpol.yaml`

THE VALIDATION TRICK

```
$ kubectl apply -f netpol-default-deny.yaml --dry-run=server # schema + admission check  
# nothing is enforced yet – safe to run against a live namespace
```

SEE IT

The same probe, three policies later

One command proves each stage: exec into a client Pod and wget the web-svc Service.

STATE	CLIENT-ALLOWED → WEB	CLIENT-BLOCKED → WEB
No policy	REACHABLE	REACHABLE
+ web-default-deny	BLOCKED	BLOCKED
+ web-allow-from-client	REACHABLE	BLOCKED

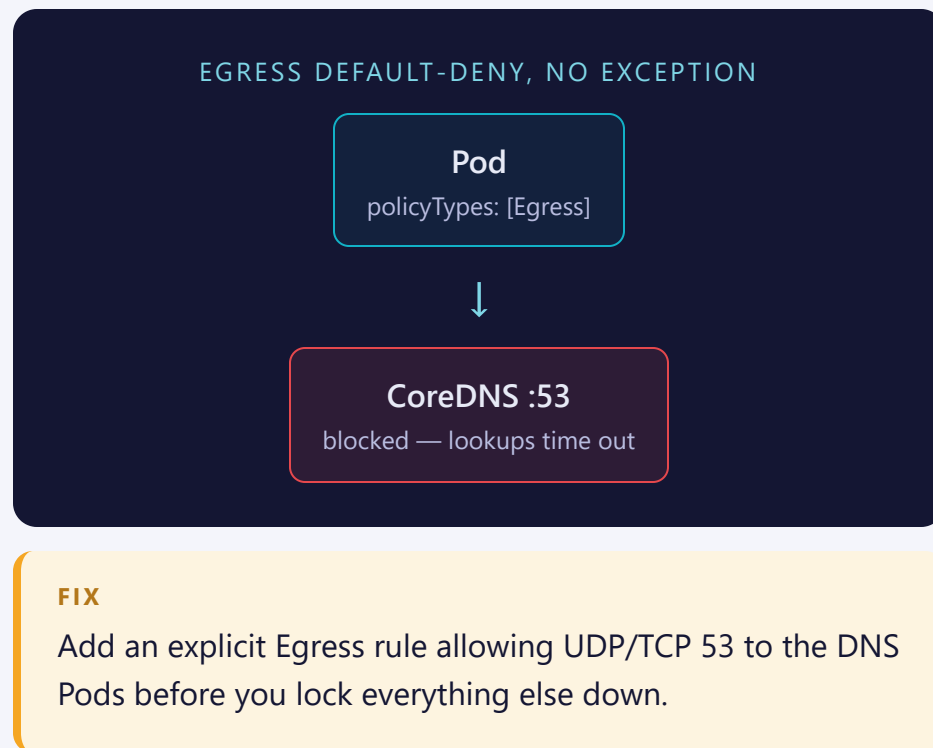
The deny sets the new default; the allow punches exactly one hole, for the Pod labelled app=client-allowed.

Default-deny egress blocks DNS too

DNS lookups are just another outbound connection — UDP/TCP **port 53** to CoreDNS. A default-deny **Egress** policy on a Pod cuts that off along with everything else.

The symptom looks nothing like a network problem: `nslookup` times out or returns `server can't find`, and app logs show connection errors to service *names* that would resolve fine if pinged by IP.

This exact failure is Lab 9.5 — DNS Problems.



Where people trip

- **"We have NetworkPolicies" ≠ isolated.** If nothing selects a Pod, it stays wide open — the flat-network default never went away for it.
- **Egress default-deny breaks DNS** unless port 53 to CoreDNS is explicitly allowed (Lab 9.5).
- **The CNI must enforce it.** Cilium and Calico do; plain Flannel accepts the object and silently does nothing.
- **Half-listed policyTypes.** Writing an egress: block but leaving policyTypes: [Ingress] means the egress rules are never applied.

Commands to keep at hand

```
$ kubectl get networkpolicy -n NS           # which policies exist
$ kubectl describe networkpolicy NAME -n NS # selector + rules
$ kubectl get pods -n NS --show-labels      # does the selector actually match?
$ kubectl exec POD -n NS -- wget -T 3 -q0- SVC # probe reachability directly
$ kubectl apply -f netpol.yaml --dry-run=server # validate before enforcing
```

Tip: "suddenly nothing resolves DNS" after a policy change → check for a default-deny Egress with no port-53 exception first.

RECAP

Network Policies in one breath

- No policy = flat network, everything reachable
- Selecting a Pod flips that direction to deny-by-default
- Policies are additive — allow rules punch holes, there's no explicit deny
- The CNI must enforce it (Cilium here); Flannel silently won't
- Egress default-deny needs an explicit DNS (port 53) allow

HANDS-ON

Now run Lab 4.7
[labs/4.7/](#)

Next: [5.1 — Volumes](#)